

Matteo Palmonari

matteo.palmonari@disco.unimib.it

Modeling Knowledge Graphs: *OWL: the Ontology Web Language #1*



dat·AI data * AI
AI * data

OWL: Outline

■ Limitations of RDFS

■ OWL Highlights

- OWL most important features

■ OWL & Description Logics

- Mathematical background supporting OWL reasoning

■ Other Relevant Features

- Some insights into existing reasoners

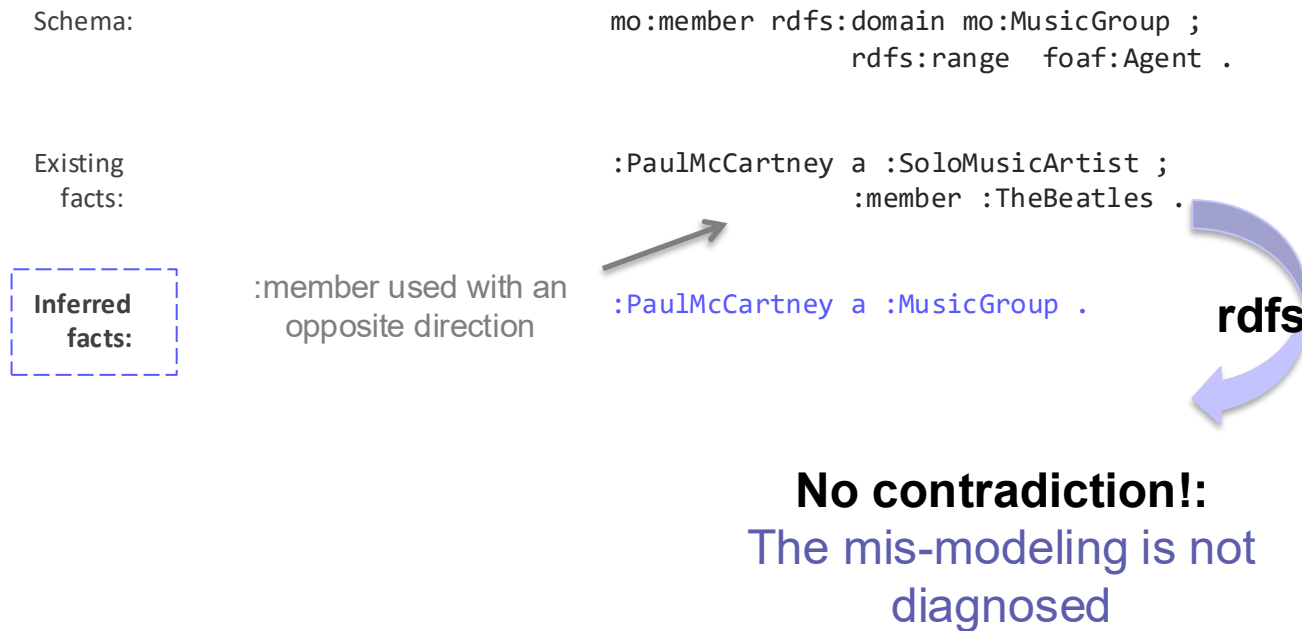


Highlights

(OWL from a more
practical point of view)

RDFS: Lack of Consistency Check

- It is possible to infer facts that seem incorrect facts, but RDFS cannot prevent this:



More limitations of RDFS

❑ By Fausto Giunchiglia*

❑ No **Disjoint** classes

- ❑ Cannot say that Graduate and PhD. Students are two disjoint classes

❑ No **transitive, inverse or symmetrical** properties

❑ No **cardinality** constraints

- ❑ Cannot say that a course is taught by at least one professor, or persons have exactly 2 parents

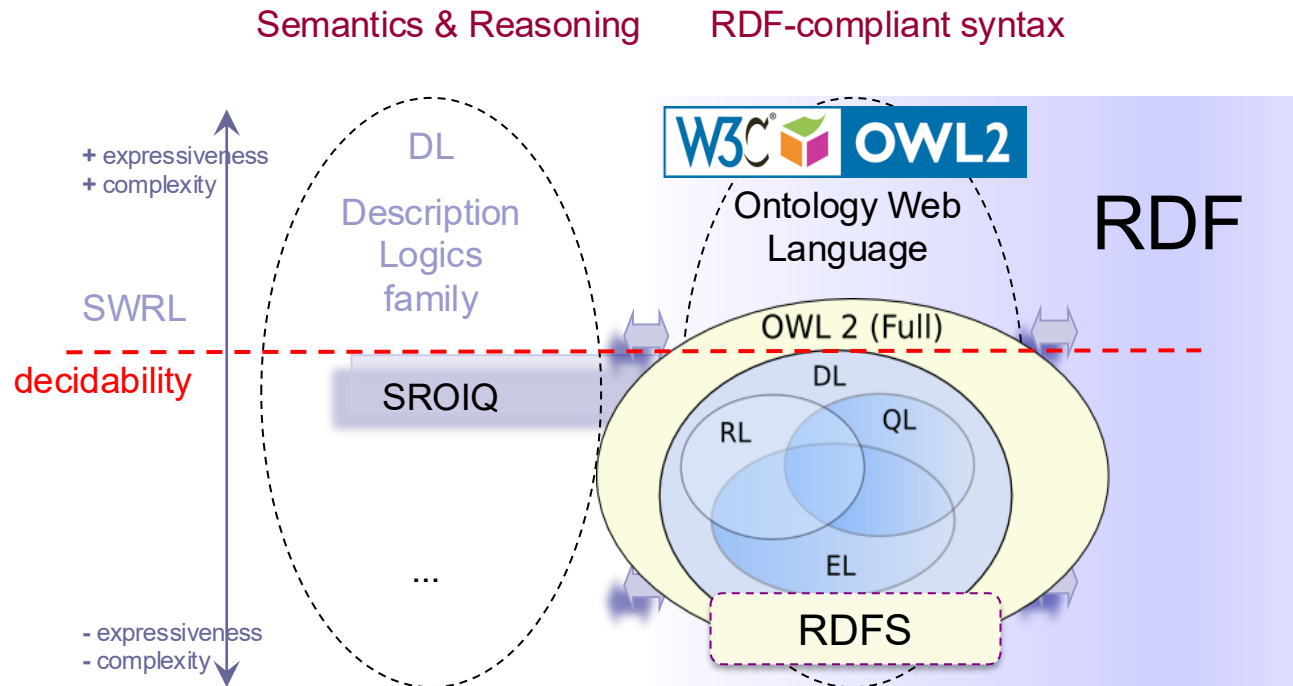
❑ No **Boolean combinations** of classes

- ❑ Sometimes we may need to build new classes by combining other classes using **union**, **intersection**, and **complement** (e.g. person is the **disjoint union** of the classes male and female)

❑ No **localised range and domain** constraints

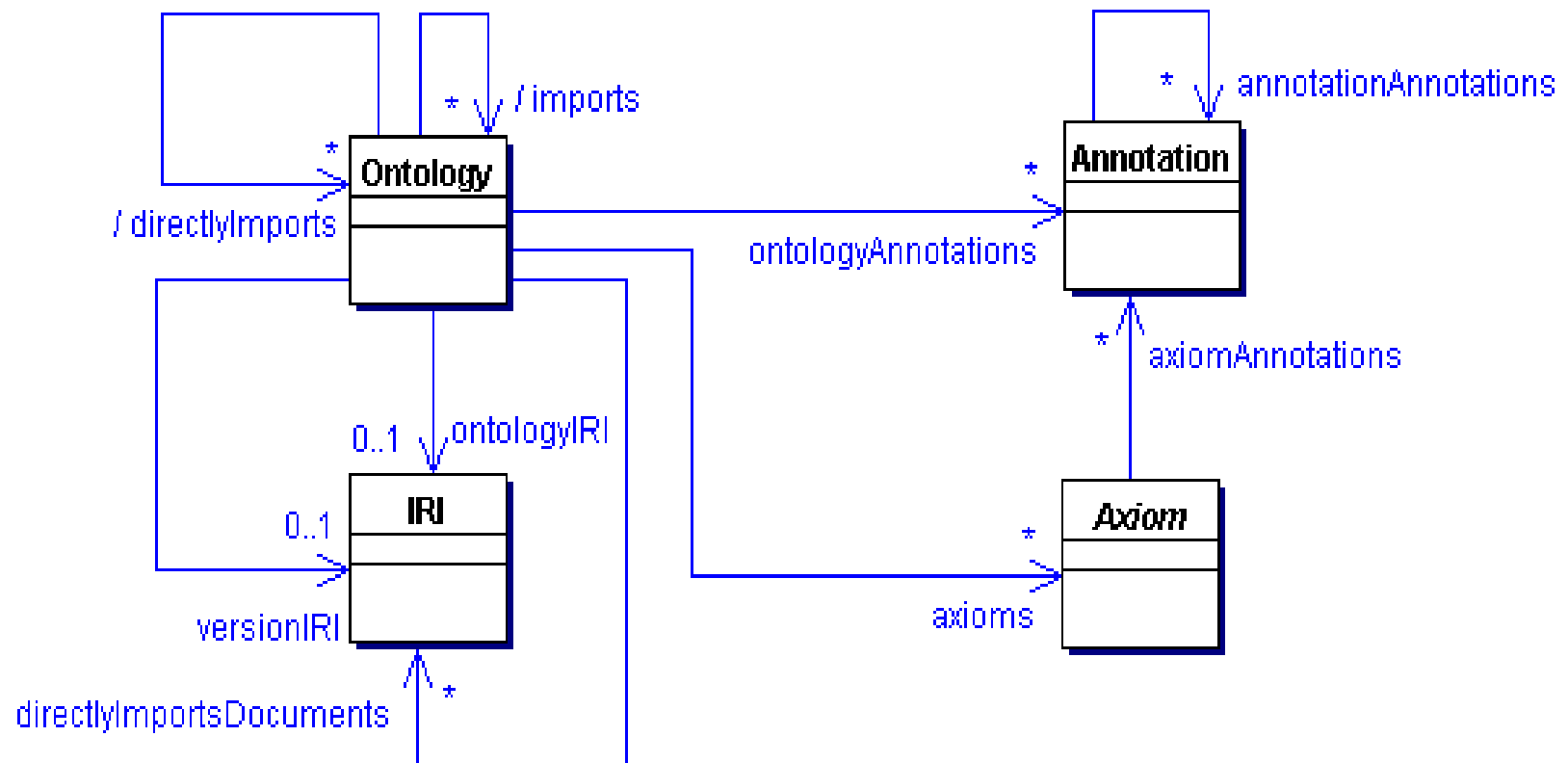
- ❑ Cannot say that the range of taughtBy is only professor when applied to professors and lecturer when applied to lecturers

Languages/models for RDF Ontologies



Vocabularies/ontologies with different expressive powers

Ontologies in OWL 2*



OWL 2 Ontology Structure

OWL 2 DL, i.e., OWL with Direct Semantics

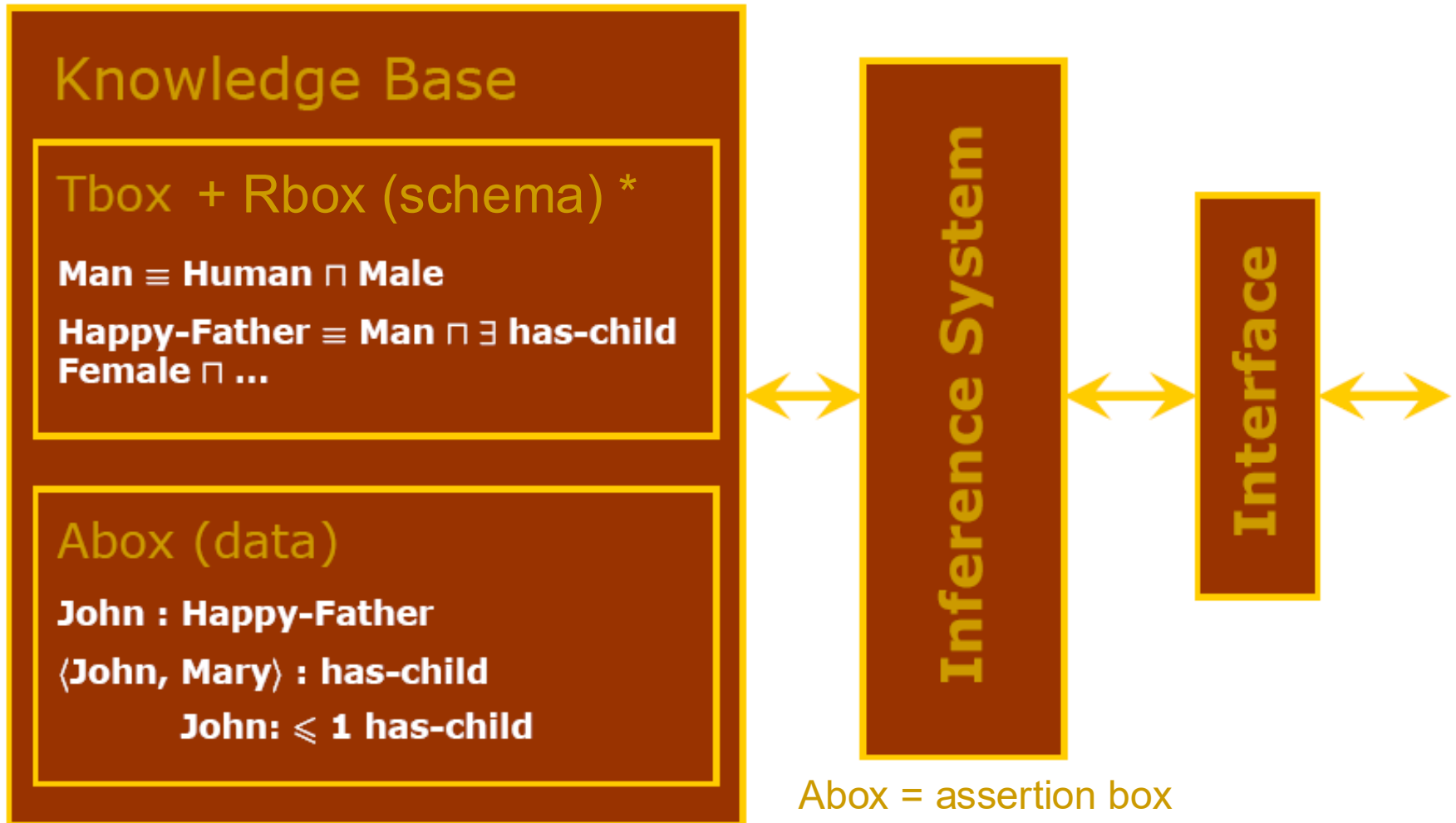
- ❑ Two alternative ways of assigning meaning to ontologies in OWL 2:
 - ❑ the direct semantics (*provides a meaning for OWL 2 in a **Description Logics** style*);
 - ❑ <https://www.w3.org/TR/2012/REC-owl2-direct-semantics-20121211/>
 - ❑ the RDF-Based Semantics (*an extension of the semantics for RDFS and is based on viewing OWL 2 ontologies as RDF graphs*)
 - ❑ <https://www.w3.org/TR/2012/REC-owl2-rdf-based-semantics-20121211/>
- ❑ The notion **OWL 2 DL** is used to refer to OWL 2 ontologies interpreted using the Direct Semantics
- ❑ The notion OWL 2 Full is used when considering the RDF-Based Semantics

A Definition from DL

Also Very Useful in OWL

Tbox = terminology box

Rbox = relation box

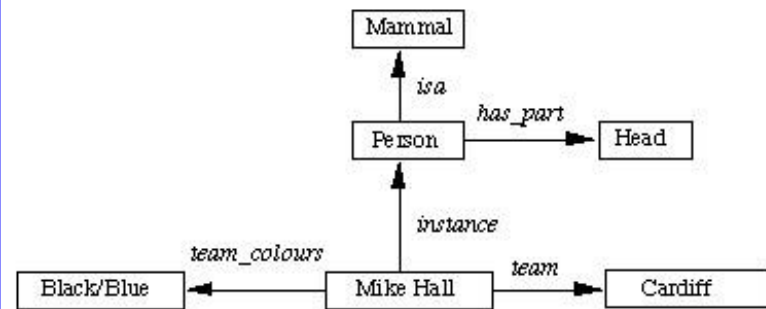


In OWL(1) we only had Tbox; for our course, it is ok to refer to "Tbox" to refer to both TBox and RBox

Representation, Semantics, Reasoning: Description Logics

Representation in a Semantic Net

The physical attributes of a person can be represented as in Fig. 2.



Precursors of DLs: semantic nets

Woman	$\equiv$	Person $\sqcap$ Female
Man	$\equiv$	Person $\sqcap$ $\neg$ Woman
Mother	$\equiv$	Woman $\sqcap$ $\exists$ hasChild.Person
Father	$\equiv$	Man $\sqcap$ $\exists$ hasChild.Person
Parent	$\equiv$	Father $\sqcup$ Mother
Grandmother	$\equiv$	Mother $\sqcap$ $\exists$ hasChild.Parent
MotherWithManyChildren	$\equiv$	Mother $\sqcap$ ≥ 3 hasChild
MotherWithoutDaughter	$\equiv$	Mother $\sqcap$ $\forall$ hasChild. $\neg$ Woman
Wife	$\equiv$	Woman $\sqcap$ $\exists$ hasHusband.Man

Fig. 2.2. A terminology (TBox) with concepts about family relationships.

MotherWithoutDaughter(MARY)	Father(PETER)
hasChild(MARY, PETER)	hasChild(PETER, HARRY)
hasChild(MARY, PAUL)	

Fig. 2.4. A world description (ABox).

Representation, Semantics, Reasoning: Description Logics

TBox (and Rbox)
axioms represent
general knowledge
about classes and
properties

ABox axioms
represent specific facts
about individuals

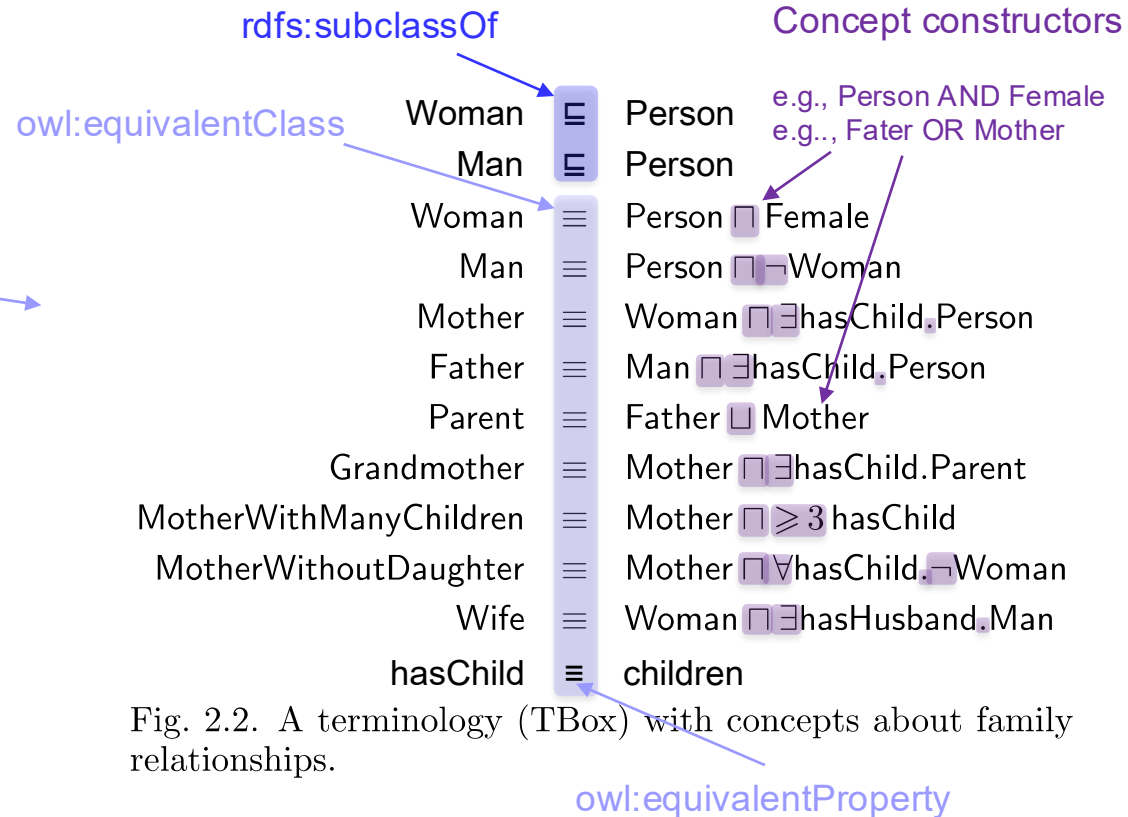


Fig. 2.2. A terminology (TBox) with concepts about family relationships.

MotherWithoutDaughter(MARY)
hasChild(MARY, PETER)
hasChild(MARY, PAUL)

Father(PETER)
hasChild(PETER, HARRY)

Fig. 2.4. A world description (ABox).

Tbox and Rbox Axiom Schemes

Axiom structure in DL and OWL syntaxes

[class] $\sqsubseteq$ [class]

[class] rdfs:subclassOf [class]

[class] $\equiv$ [class]

[class] $\text{owl:equivalentClass}$ [class]

[property] $\sqsubseteq$ [property]

[property] $\text{rdfs:subpropertyOf}$ [property]

[property] $\equiv$ [property]

[property] $\text{owl:equivalentProperty}$ [property]

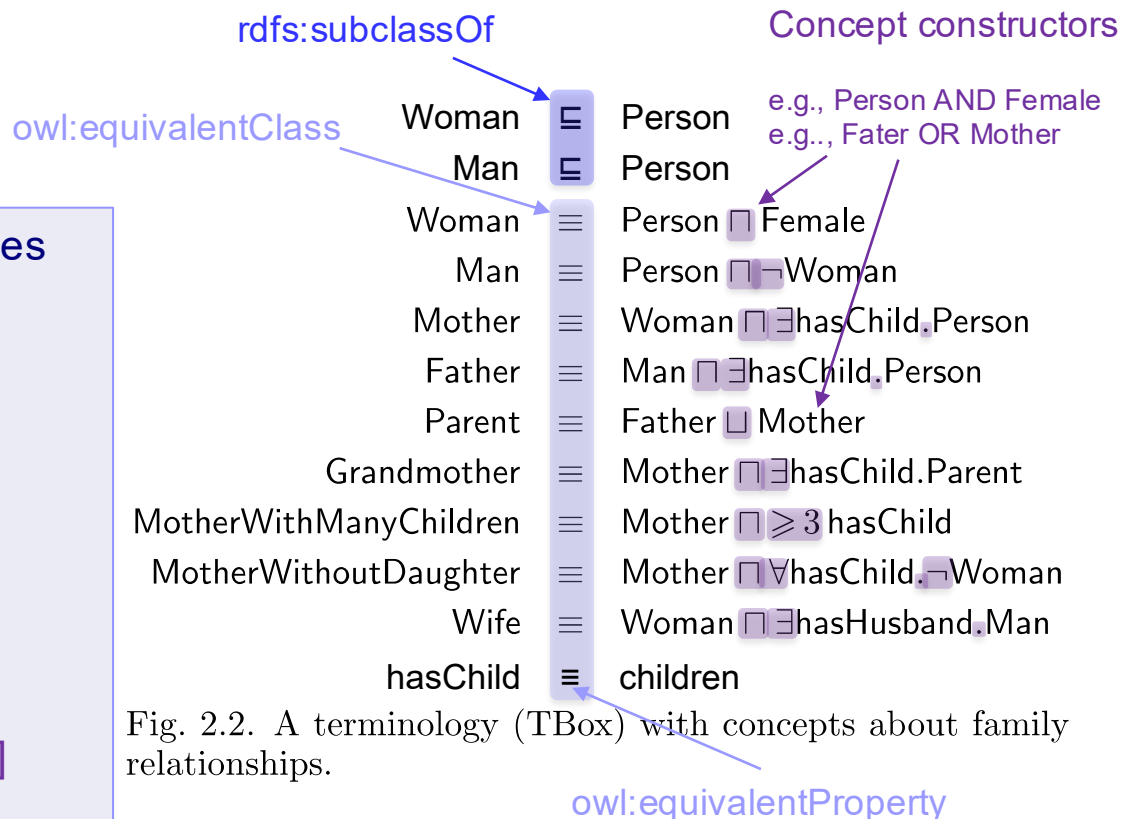


Fig. 2.2. A terminology (TBox) with concepts about family relationships.

On the **right**: named classes/properties, or complex classes/properties built using constructors, e.g., $\text{Person} \sqcap \text{Female}$, $\text{inverseOf}(\text{children})$

On the **left**: typically named classes/properties, e.g., Woman, hasChild

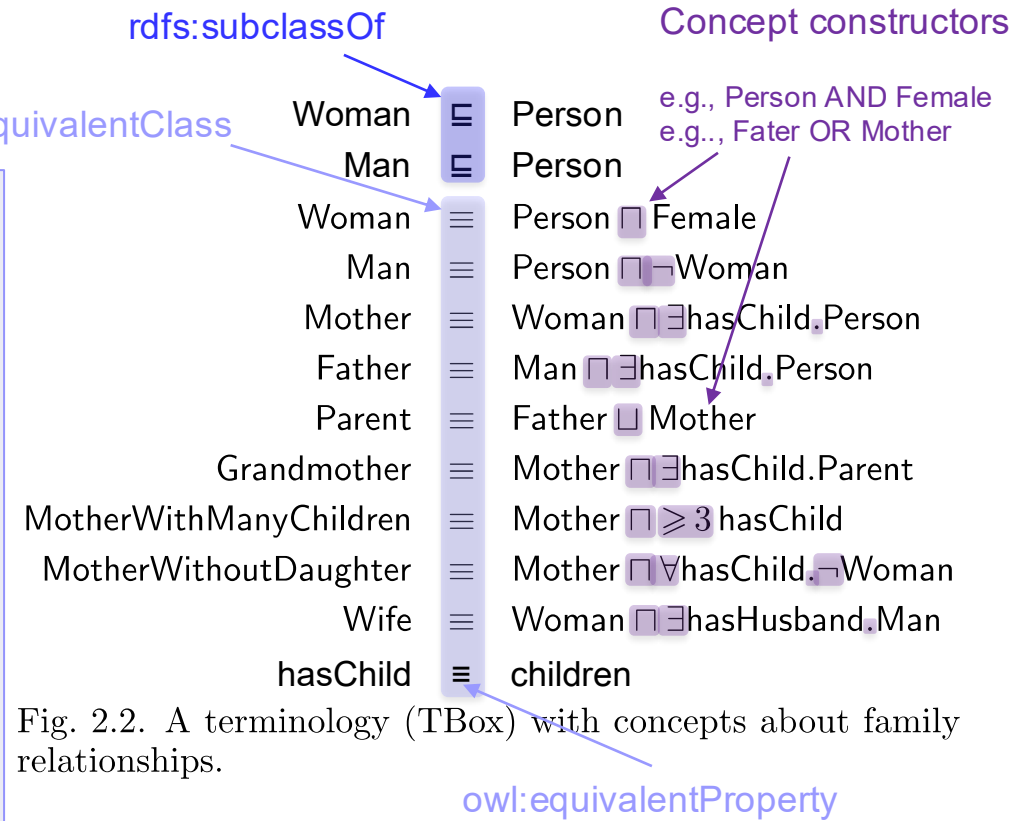
Remark!

Sometimes it's easier to interpret consent

Sometimes it's easier to interpret concept constructors by considering the axioms in which they appear ...

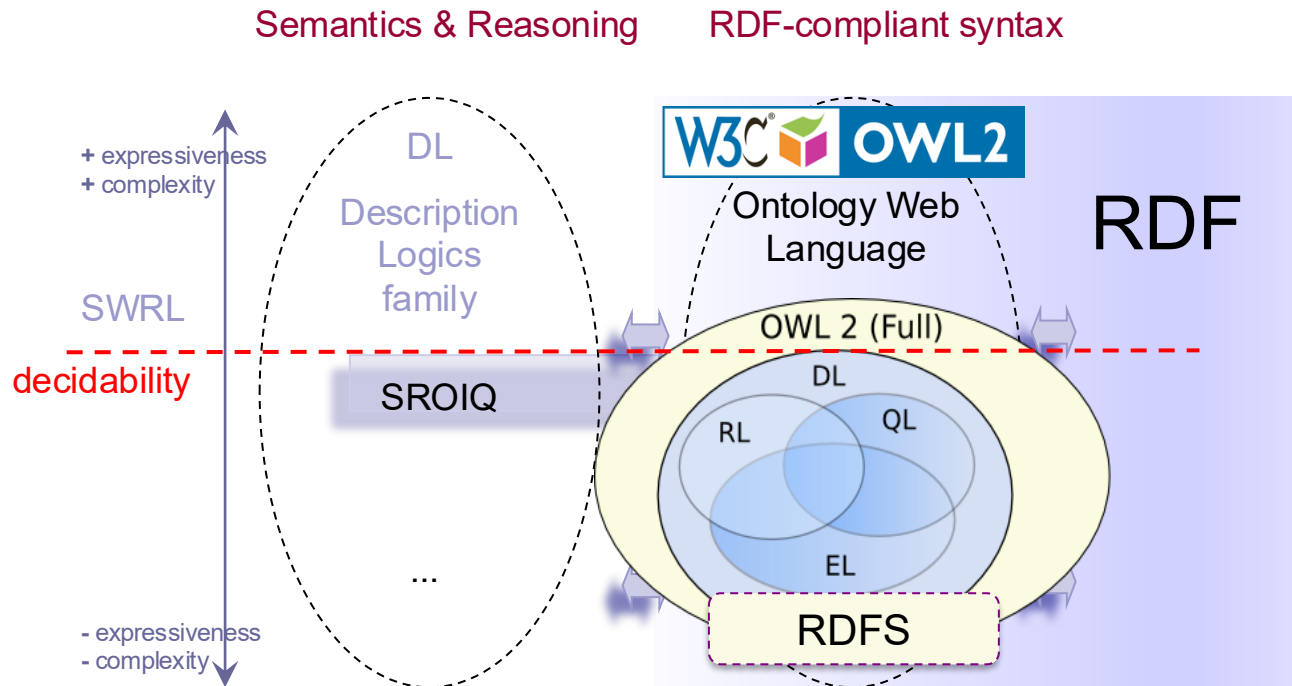
Example*:

- $\forall \text{ producer.MovieProducer}$
 - «*the class of instances for which the property producer has only values of class MovieProducer*»
- $\text{Movie} \sqsubseteq \forall \text{ producer.MovieProducer}$
 - «*all producers of Movie(s) must be of class MovieProducer*»



- * we'll explain this axiom/constructor later, now I only want to show that an axiom can be interpreted more intuitively than a complex class it uses

Languages/models for RDF Ontologies



We may also have something in the middle,

- e.g., RDFS++:
RDFS with some OWL axiom

Vocabularies/ontologies with different expressive powers

Introduction to OWL

- Provides more ontological constructs and avoids some of the potential confusion in RDFS
- OWL 2 is divided into sub-languages denominated *profiles*:
 - OWL 2 EL: Limited to basic classification, but with polynomial-time reasoning
 - OWL 2 QL: Designed to be translatable to relational database querying
 - OWL 2 RL: Designed to be efficiently implementable in rule-based systems
- Most triple stores concentrate on the use of RDFS with a subset of OWL features, called OWL-Horst or RDFS++

More restrictive
than OWL DL

These slides

Class Axioms

Axioms declare general statements about concepts which are used in logical inference (reasoning). Class axioms:

- **Sub-class** relationship (from RDF Schema)
- **Equivalent** relationship: classes have the same individuals

```
:Musician owl:equivalentClass :MusicArtist .
```

- **Disjointness**: classes have no shared individuals

```
:SoloMusicArtist owl:disjointWith :MusicGroup .
```

Equivalent Classes, Definitions and Boolean Classifiers

rdfs:subClassOf

Turtle syntax
:Woman a owl:Class ;
 rdfs:subClassOf [
 a owl:Class ;
 owl:intersectionOf (:Person :Female)
] .

DL syntax
Woman $\sqsubseteq$ Person $\sqcap$ Female

Interpretation
If x is a Woman,
then
- she is a Person
- she is a Female

owl:equivalentClass

:Woman a owl:Class ;
 owl:equivalentClass [
 a owl:Class ;
 owl:intersectionOf (:Person :Female)
] .

Woman $\equiv$ Person $\sqcap$ Female

... equivalent to ...

Woman $\sqsubseteq$ Person $\sqcap$ Female

Person $\sqcap$ Female $\sqsubseteq$ Woman

If x is a Woman,
then
- she is a Person
- she is a Female

If
- x is a Person
- x is a Female
Then she is a Woman

Equivalent Classes, Definitions and Boolean Classifiers

■ subClassOf

- E.g., $Woman \sqsubseteq Person \sqcap Female$
- Specifies necessary conditions for instances of the class on the left

■ equivalenClass

- E.g., $Woman \equiv Person \sqcap Female$
- Specifies necessary **and sufficient** conditions for instances of the class on the left
 - Being a Person and a Female is sufficient to classify something as a Woman
- Full-fledged concept definitions
- Behave as Boolean classifiers

Class Axioms

Axioms declare general state in logical inference (reasoning)

- **Sub-class** relationship
- **Equivalent** relationship: classes have the same individuals

Woman	≡	Person $\sqcap$ Female
Man	≡	Person $\sqcap$ $\neg$ Woman
Mother	≡	Woman $\sqcap$ $\exists$ hasChild.Person
Father	≡	Man $\sqcap$ $\exists$ hasChild.Person
Parent	≡	Father $\sqcup$ Mother
Grandmother	≡	Mother $\sqcap$ $\exists$ hasChild.Parent
MotherWithManyChildren	≡	Mother $\sqcap$ ≥ 3 hasChild
MotherWithoutDaughter	≡	Mother $\sqcap$ $\forall$ hasChild. $\neg$ Woman
Wife	≡	Woman $\sqcap$ $\exists$ hasHusband.Man

Fig. 2.2. A terminology (TBox) with concepts about family relationships.

```
:Musician owl:equivalentClass :MusicArtist .
```

Logic-based classifiers based on
concept definitions

Jane is a Woman
Jane hasChild Bill
Bill is a Person



Jane is a Mother

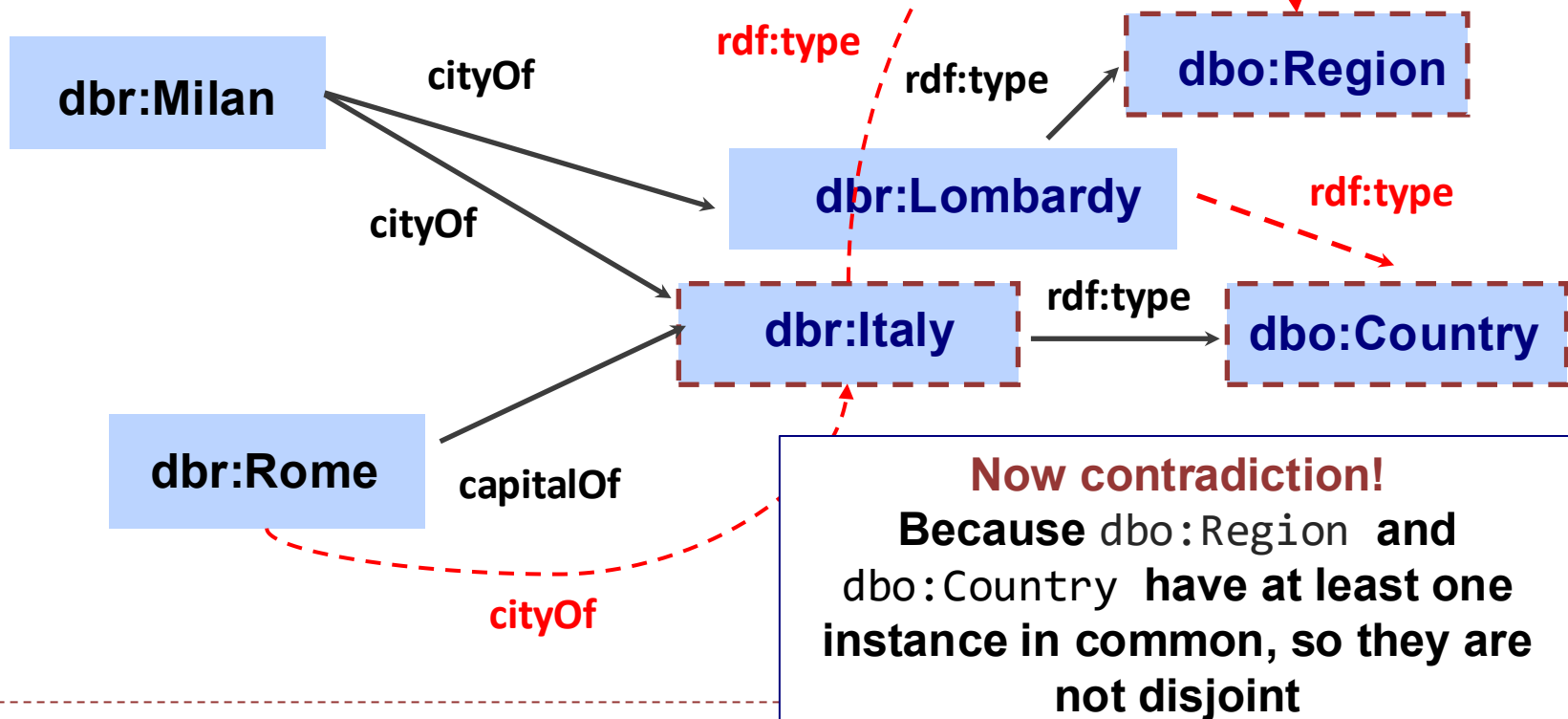
have no shared individuals

```
disjointWith :MusicGroup .
```

Adding Consistency Check to RDFS (2)

Attention to unintended and
undesired consequences!
inconsistencies

```
capitalOf rdfs:domain Capital .  
capitalOf rdfs:range dbo:Country .  
cityOf rdfs:range dbo:Country .  
cityOf rdfs:range dbo:Region .
```



dbo:Region owl:disjointWith dbo:Country

Axioms about Individuals

OWL Individuals represent instances of classes. They are related to their class by the **rdf:type** property

- We can state that two individuals are the same

RDFS++

```
<artist/ba550d0e-adac-4864-b88b-407cab5e76af#_> owl:sameAs  
dbpedia:PaulMcCartney .
```

- We can state that two individuals are different

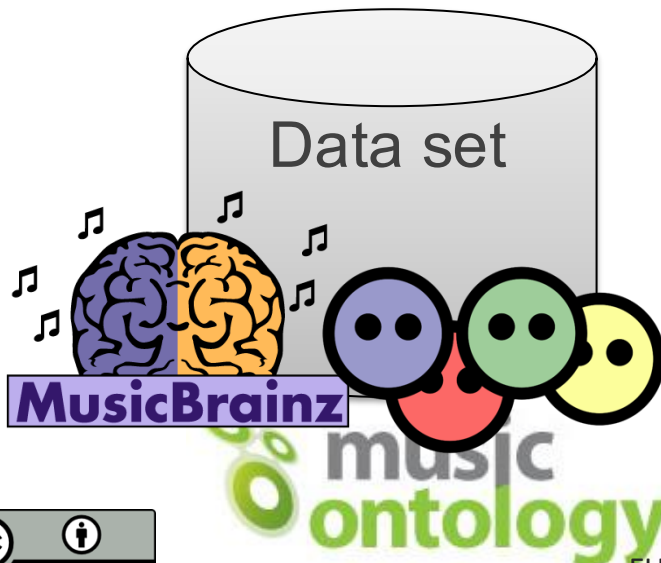
```
:TheBeatles_band owl:differentFrom :TheBeatles_Tvseries .
```

Reasoning for Linked Data Integration

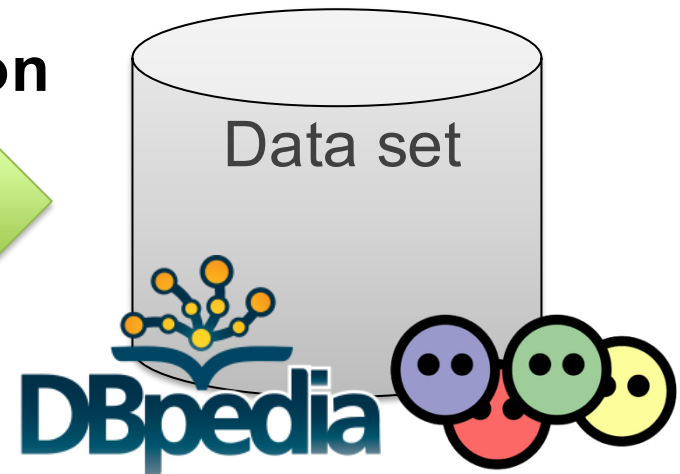
same

mo:b10bbbfc-cf9e-42e0-be17-e2c3e1d2600d ↔ dbpedia:The_Beatles
foaf:name The Beatles;
mo:member
mo:ba550d0e-adac-4864-b88b-407cab5e76af;
mo:member
mo:4d5447d7-c61c-4120-ba1b-d7f471d385b9;
mo:member
mo:42a8f507-8412-4611-854f-926571049fa0;
mo:member
mo:300c4c73-33ac-4255-9d57-4e32627f5e13.

dbpedia-ont:origin dbpedia:Liverpool;
dbpedia-ont:genre dbpedia:Rock_music;
foaf:depiction  .



Integration



OWL Properties

OWL distinguishes between two types of properties:

- **OWL ObjectProperties:** resources as values
- **OWL DatatypeProperties:** literals as values

```
:plays      rdf:type      owl:ObjectProperty;  
            rdfs:domain  :Musician;  
            rdfs:range   :Instrument .
```

```
:hasMembers rdf:type      owl:DatatypeProperty;  
            rdfs:domain  :MusicGroup  
            rdfs:range   xsd:int .
```

Property Axioms

- Property axioms include those from RDF Schema
- OWL allows for property **equivalence**. Example:

```
dbpedia-ont:bandMember owl:equivalentProperty mo:member.
```

- OWL allows for property **disjointness**. Example:

```
dbpedia-ont:length owl:propertyDisjointWith mo:duration.
```

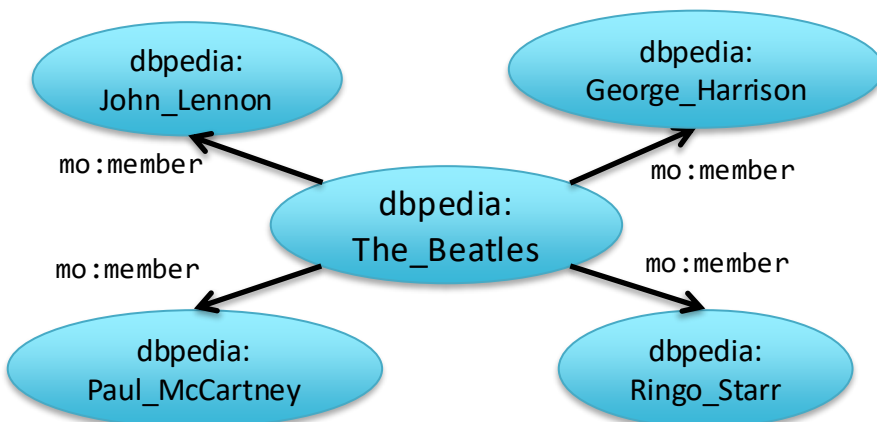
Property Axioms

- Property axioms include those from RDF Schema
- OWL allows for property **equivalence**. Example:

```
dbpedia-ont:bandMember owl:equivalentProperty mo:member.
```

Query:

```
SELECT ?x {dbpedia:The_Beatles  
           dbpedia-ont:bandMember ?x.}
```



Result set:

?x

Result set with inference:

?x

dbpedia:Paul_McCartney

dbpedia:John_Lennon

dbpedia:Ringo_Starr

dbpedia:George_Harrison

Property Axioms (2)

OWL allows the definition of property characteristics to infer new facts relating to instances and their properties

- Symmetry
- Inverse
- Transitivity
- Functional
- Inverse Functional

RDFS++

Property Axioms (2)

OWL allows the definition of property characteristics to infer new facts relating to instances and their properties

- Symmetry

$\langle p \text{ rdf:type } \text{owl:SymmetricProperty} \rangle$
 $\langle x \text{ } p \text{ } y \rangle \text{ iff } \langle y \text{ } p \text{ } x \rangle$

- Inverse

$\langle p \text{ owl:inverseOf } q \rangle$
 $\langle x \text{ } p \text{ } y \rangle \text{ iff } \langle y \text{ } q \text{ } x \rangle$

- Transitivity

- Functional

RDFS++

- Inverse Functional

- Inferred facts in OWL, **beyond** the entailment rules implemented by **RDFS**

```
Existing    :PaulMcCartney foaf:made :Yesterday ;
fact:      foaf:knows :RingoStarr .
```

```
NOW      :Yesterday dc:creator :PaulMcCartney.
inferred: :RingoStarr foaf:knows :PaulMcCartney .
```

Property Axioms (2)

OWL allows the definition of property characteristics to infer new facts relating to instances and their properties

- Symmetry

$\langle p \text{ rdf:type } \text{owl:SymmetricProperty} \rangle$
 $\langle x \text{ } p \text{ } y \rangle \text{ iff } \langle y \text{ } p \text{ } x \rangle$

- Inverse

$\langle p \text{ owl:inverseOf } q \rangle$
 $\langle x \text{ } p \text{ } y \rangle \text{ iff } \langle y \text{ } q \text{ } x \rangle$

- Transitivity

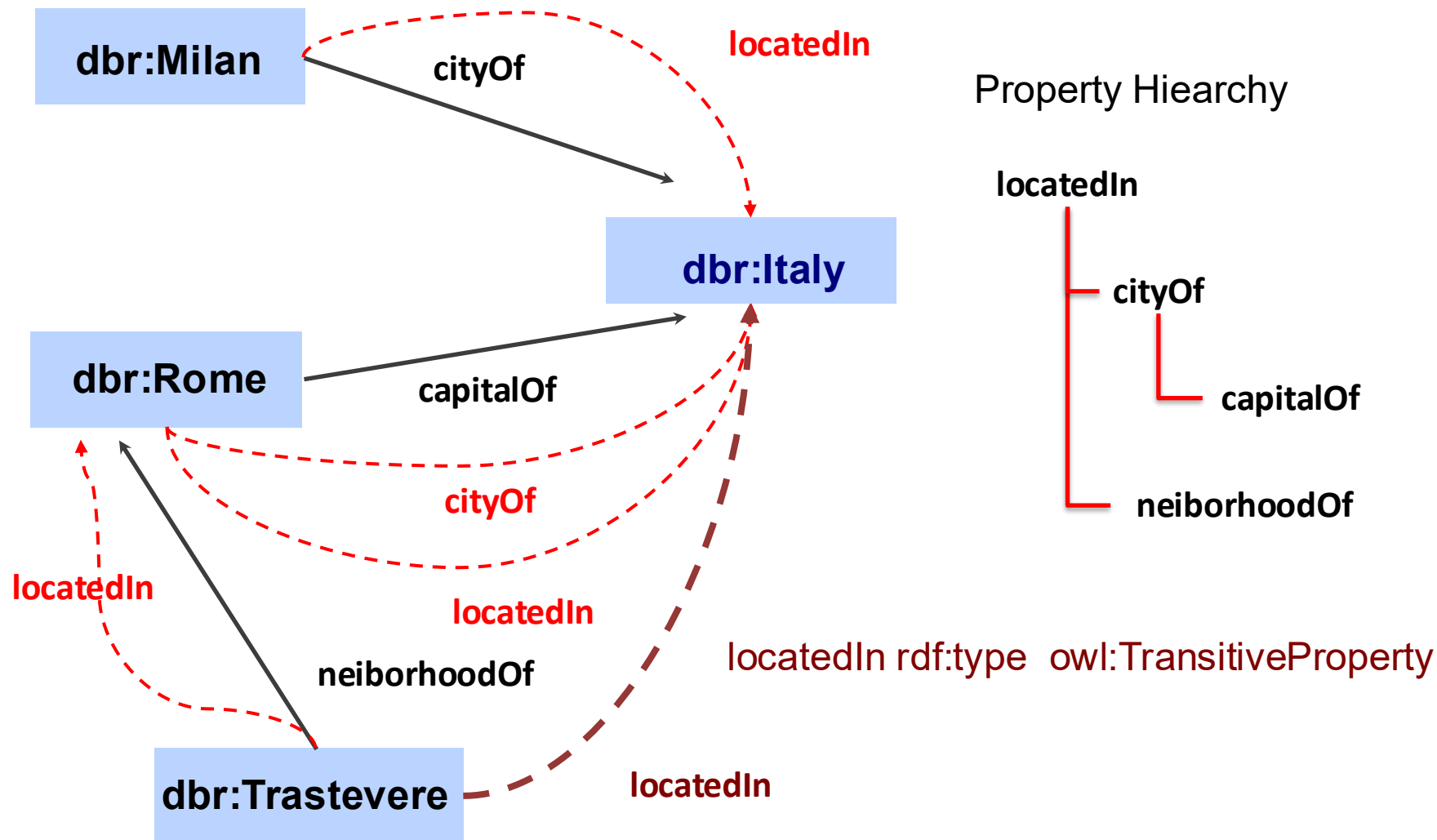
$\langle p \text{ rdf:type } \text{owl:TransitiveProperty} \rangle$
If ($\langle x \text{ } p \text{ } y \rangle$ and $\langle y \text{ } p \text{ } z \rangle$) then $\langle x \text{ } p \text{ } z \rangle$

- Functional

RDFS++

- Inverse Functional

Minimal Reasoning with RDFS



Property Axioms (2)

OWL allows the definition of property characteristics to infer new facts relating to instances and their properties

- Symmetry

< p rdf:type owl:SymmetricProperty >
 $\langle x \ p \ y \rangle \text{ iff } \langle y \ p \ x \rangle$

- Inverse

< p owl:inverseOf q >
 $\langle x \ p \ y \rangle \text{ iff } \langle y \ q \ x \rangle$

- Transitivity

< p rdf:type owl:TransitiveProperty >
If ($\langle x \ p \ y \rangle$ and $\langle x \ p \ z \rangle$) then $\langle x \ p \ z \rangle$

- Functional

< p rdf:type owl:FunctionalProperty >
If ($\langle x \ p \ y \rangle$ and $\langle x \ p \ z \rangle$) then $\langle y \ owl:sameAs \ z \rangle$

*Functional properties represent keys and trees
(one unique value is expected)*

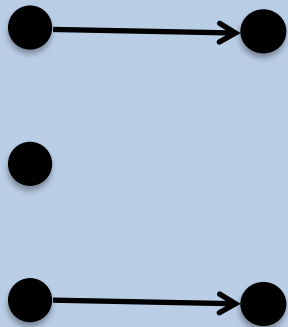
RDFS++

- Inverse Functional

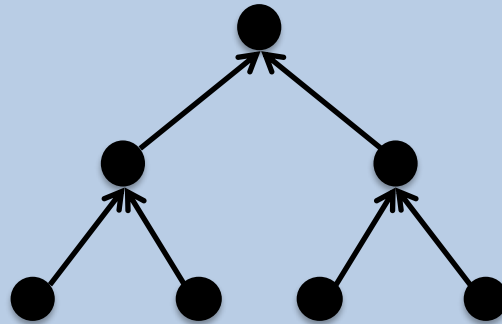
< p rdf:type owl:InverseFunctionalProperty >
If ($\langle x \ p \ z \rangle$ and $\langle y \ p \ z \rangle$) then $\langle x \ owl:sameAs \ y \rangle$

*Inverse functional properties represent trees
(one unique ancestor is expected)*

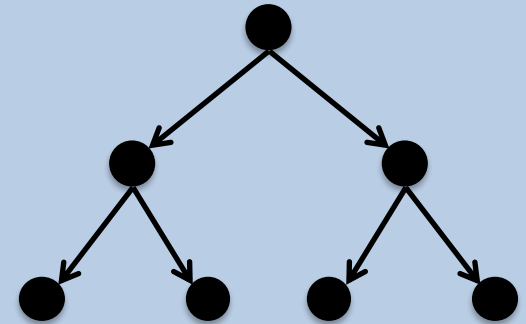
Property Axioms (2)



Functional



Functional



Inverse Functional

- Functional

`< p rdf:type owl:FunctionalProperty >`
If (`< x p y >` and `< x p z >`) then `< y owl:sameAs z >`

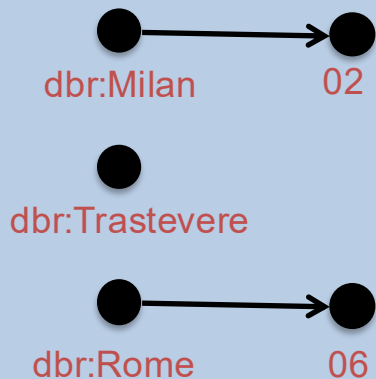
*Functional properties represent keys and trees
(one unique value is expected)*

- Inverse Functional

`< p rdf:type owl:InverseFunctionalProperty >`
If (`< x p z >` and `< y p z >`) then `< x owl:sameAs y >`

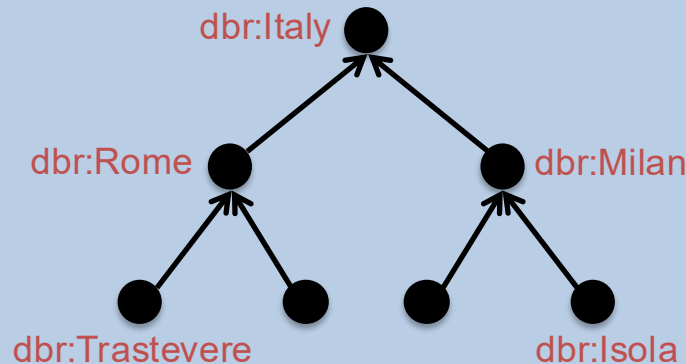
*Inverse functional properties represent trees
(one unique ancestor is expected)*

Property Axioms (2)



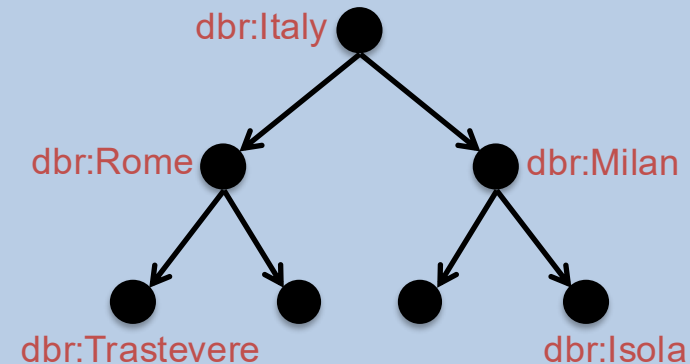
Functional

e.g. `dbo:areaCode`



Functional

e.g. `locatedIn`



Inverse Functional

e.g. `contains`

`< p rdf:type owl:FunctionalProperty >`

If (`< x p y >` and `< x p z >`) then `< y owl:sameAs z >`

*Functional properties represent keys
(one unique value is expected)*

*Inverese functional properties represent trees
(one unique ancestor is expected)*

`< p rdf:type owl:InverseFunctionalProperty >`

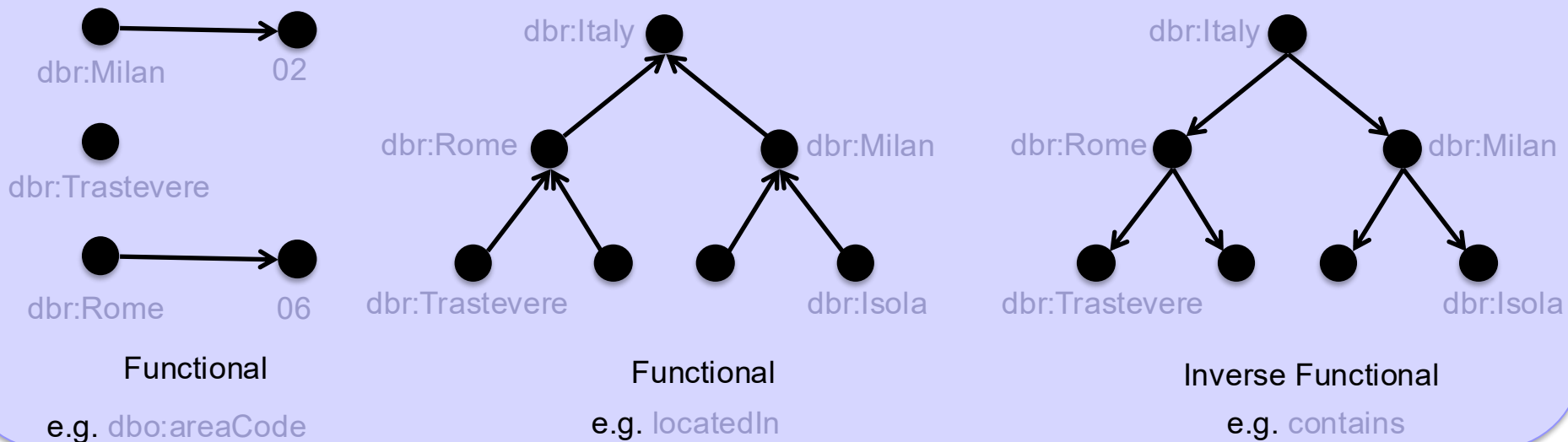
If (`< x p z >` and `< y p z >`) then `< x owl:sameAs y >`

• Functional

RDFS++

• Inverse Functional

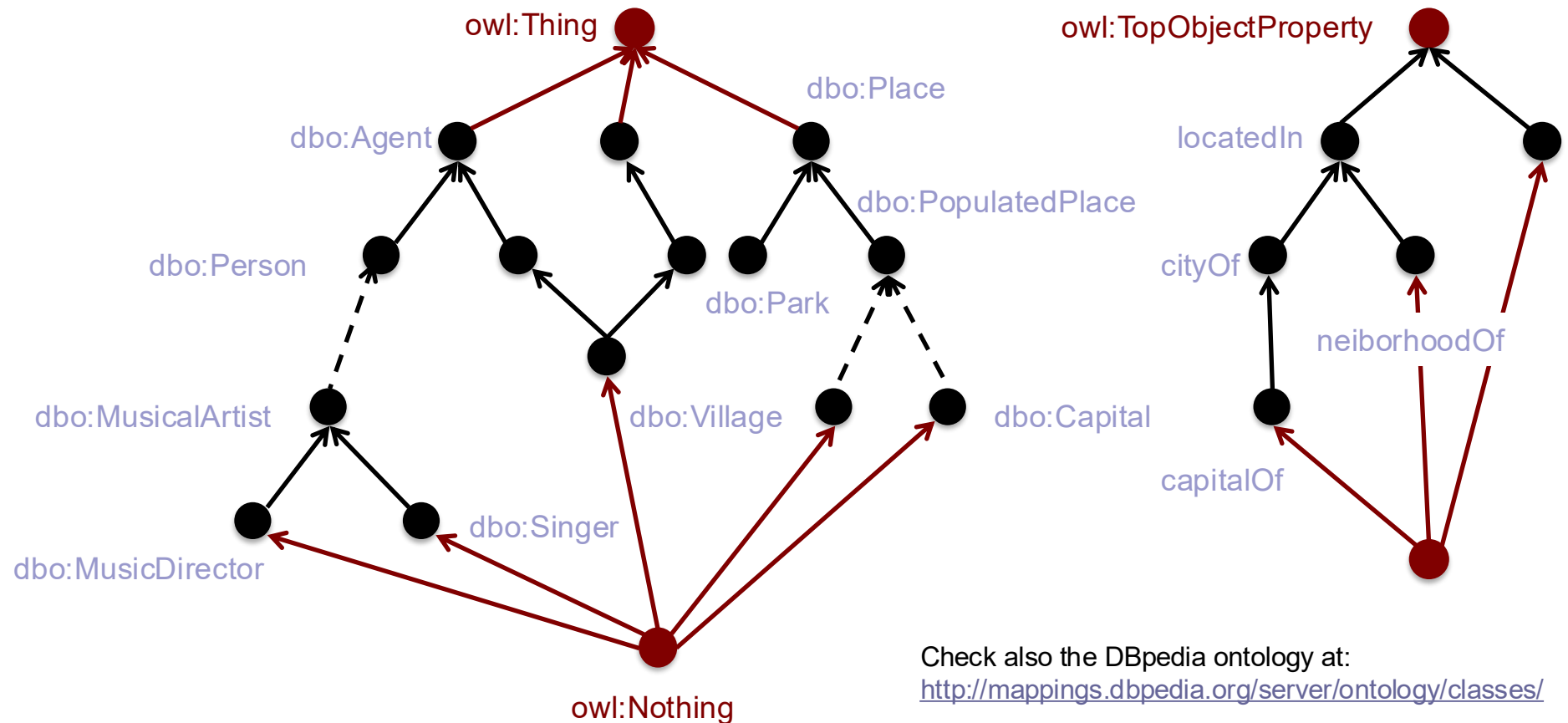
Interaction among Property Axioms



Attention to implications:

- `locatedIn` can be specified to be the inverse of `contains`
- Specifying `locatedIn` as **functional** and **transitive** at the same time lead to an inconsistency (same as for inverse functional)
 - Possible solution: add a `transitiveLocatedIn` as superproperty

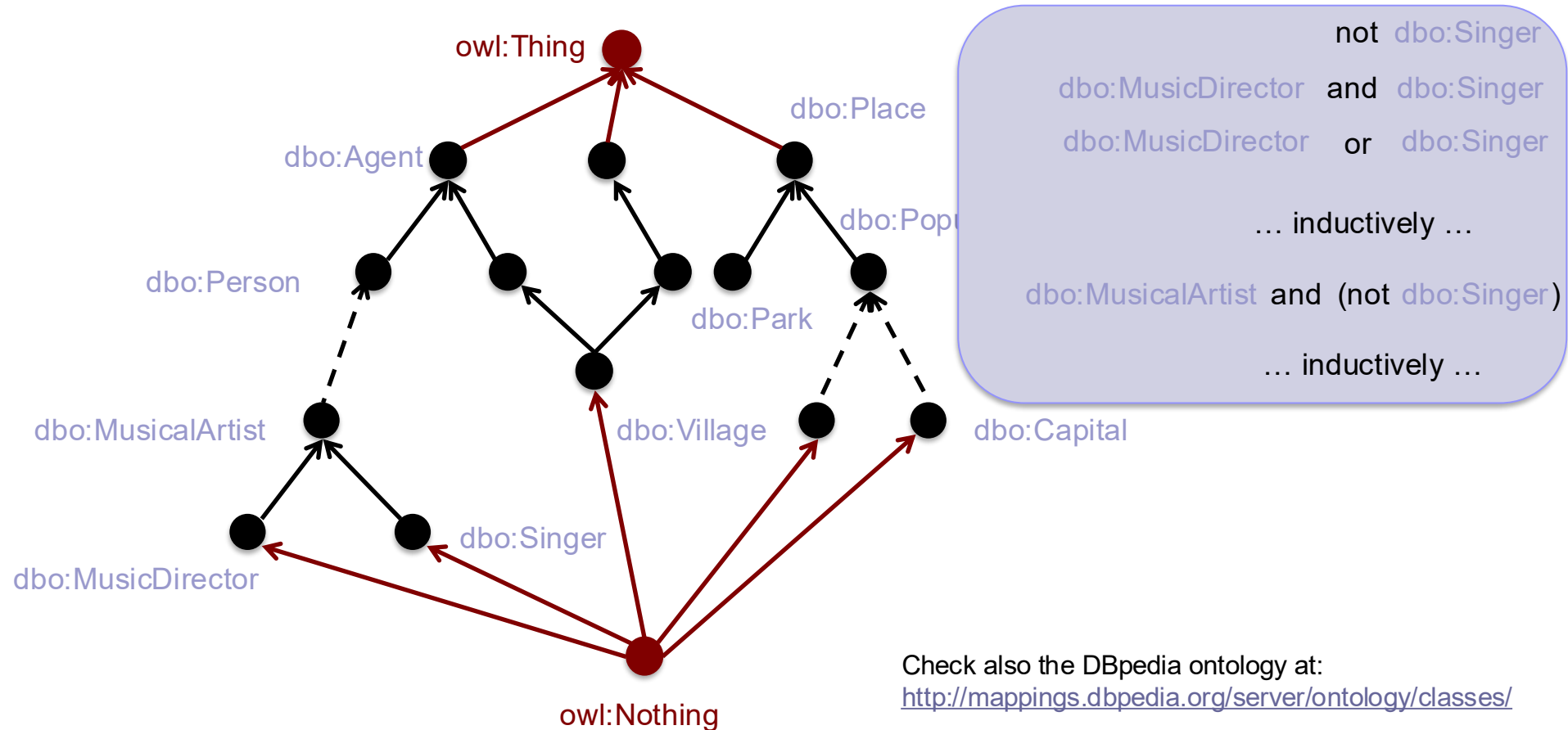
Structure of Class / Property Hierarchies



Check also the DBpedia ontology at:
<http://mappings.dbpedia.org/server/ontology/classes/>

- Class / property lattices (partially ordered sets with top / bottom)
 - all entities are instance of owl:Thing
 - owl:Nothing is the empty class (~ lattices in set theory)
- Classes (and props) may have multiple direct super classes (props)

Boolean Combinations (Class Lattice!)



- Class / property lattices (partially ordered sets with top / bottom)
 - all entities are instance of owl:Thing
 - owl:Nothing is the empty class (~ lattices in set theory)
- Classes (and props) may have multiple direct super classes (props)

More Advanced Features

■ Cardinality constraints

- a group is composed by at least 2 musicians
- a duo is composed by exactly 2 musicians

■ Localised range and domain constraints

- Universal restrictions: the range of `producer` is only `MusicArtist` when applied to records and `MovieProducer` when applied to movies
- ... similarly for existential restrictions...

■ Property composition

- `musicalArtist` (between a single and an artist) is equivalent to the composition of `album` (between a single and an album) and `artist` (between an album and an artist)

■ ...

... understanding these features is easier with Description Logics ...

QUICK

YES

SKIP

Cardinality Restrictions #1

```
:MusicGroup a owl:Class ;  
  rdfs:subClassOf [
```

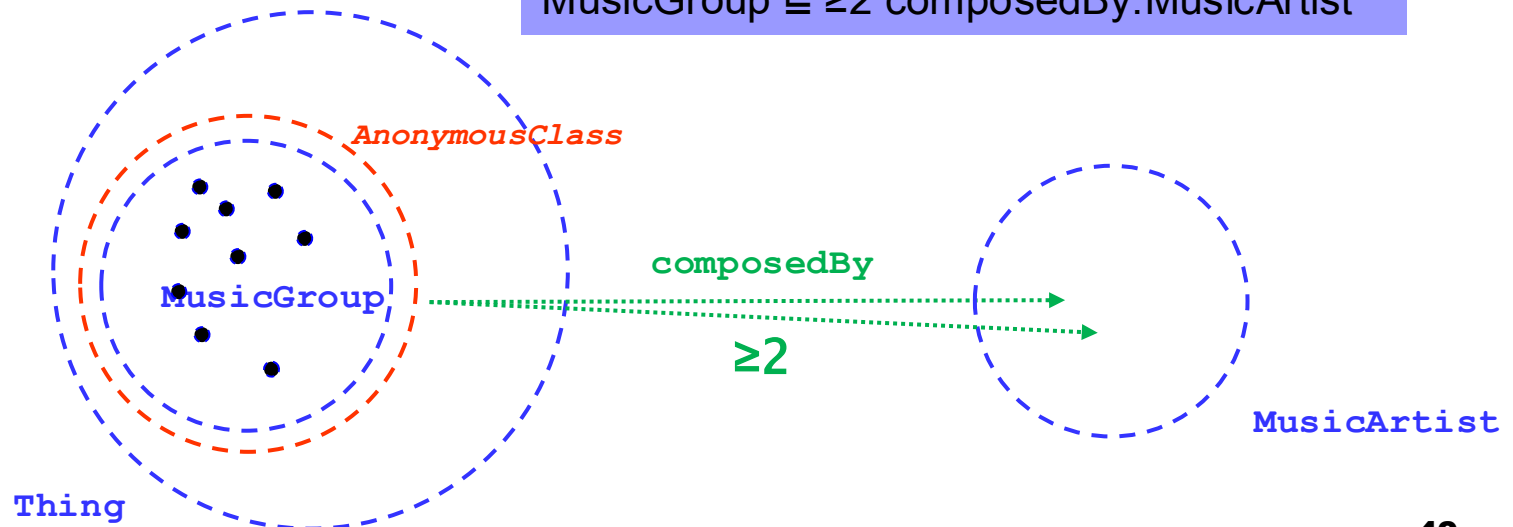
```
  a owl:Restriction ;  
  owl:onProperty :composedBy ;  
  owl:minQualifiedCardinality 2 ;  
  owl:onClass :MusicArtist .  
]
```

Not very important, but:

[] refers to an anonymous class, identified by a blank node ([] notation in Turtle)

A group is composed by *at least two* music artists

$\text{MusicGroup} \sqsubseteq \geq 2 \text{ composedBy. MusicArtist}$



Cardinality Restrictions #2

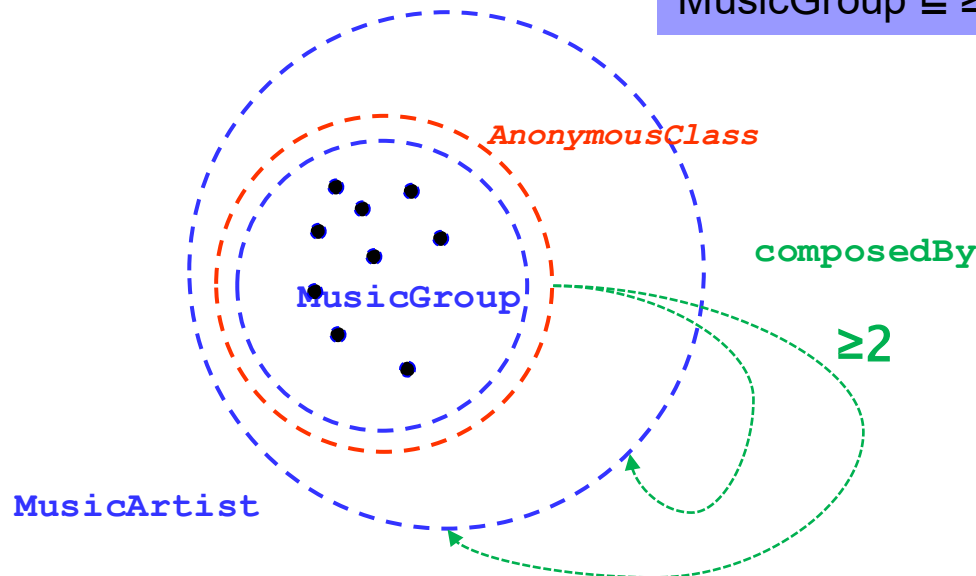
```
:MusicGroup a owl:Class ;  
  rdfs:subClassOf MusicArtist, [
```

```
  a owl:Restriction ;  
  owl:onProperty :composedBy ;  
  owl:minQualifiedCardinality 2 ;  
  owl:onClass :MusicArtist .  
]
```

QUICK

A group is composed by *at least two* music artists

$\text{MusicGroup} \sqsubseteq \geq 2 \text{ composedBy. MusicArtist}$



Cardinality Restrictions #3

```
:MusicGroup a owl:Class ;  
  rdfs:subClassOf MusicArtist, [
```

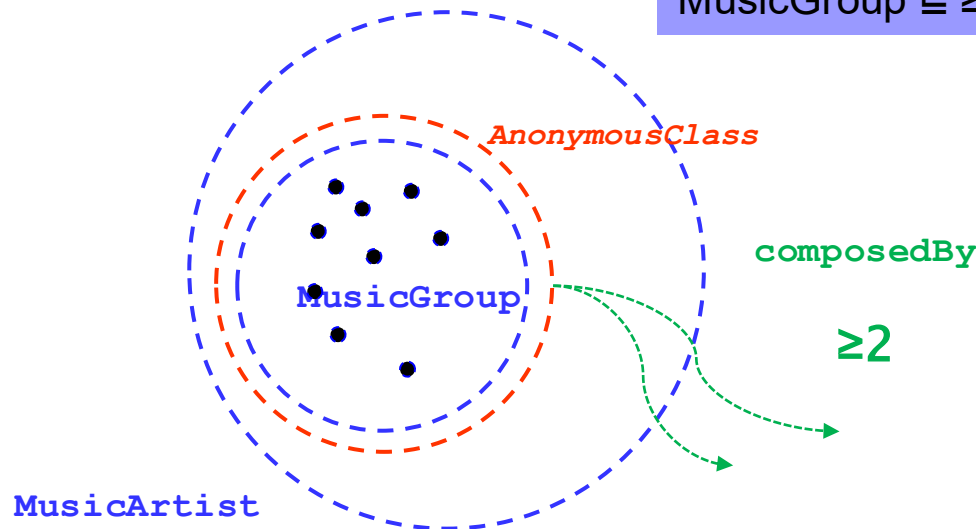
```
  a owl:Restriction ;  
  owl:onProperty :composedBy ;  
  owl:minCardinality 2 ;
```

```
] .
```

QUICK

A group is composed by *at least two* things

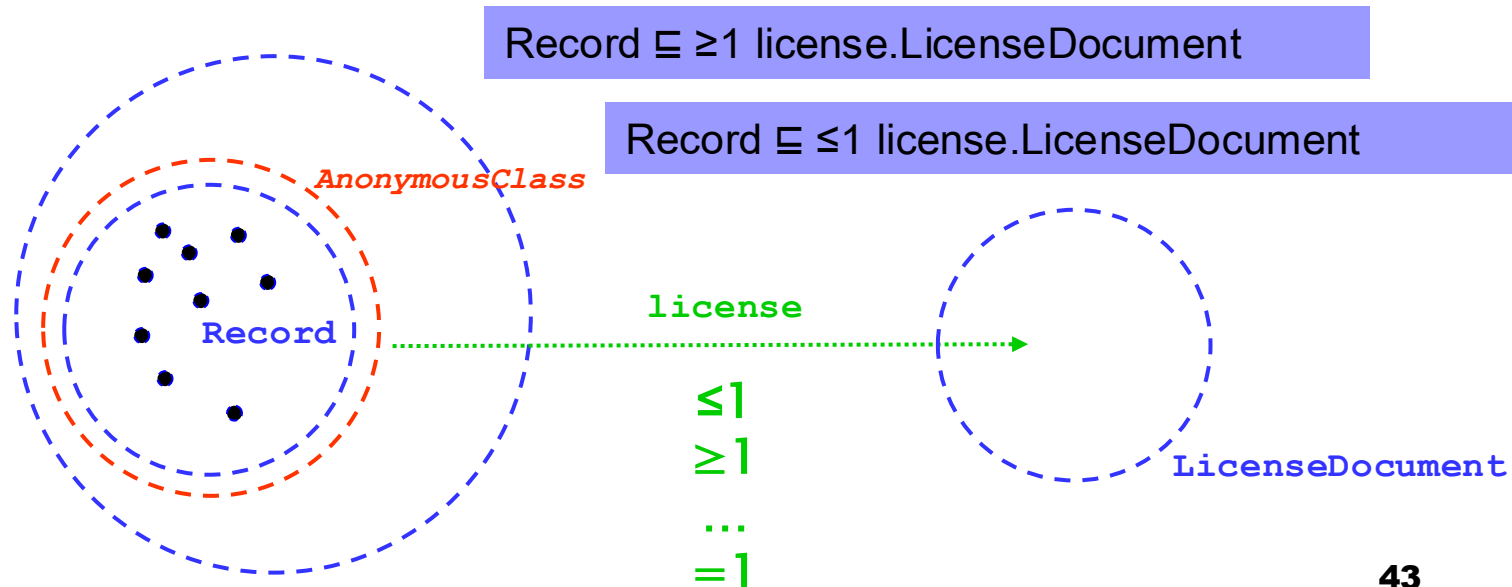
MusicGroup $\sqsubseteq \geq 2$ composedBy



Cardinality Restrictions #3

QUICK

A record has *at least one* license document attached
A record has *at most one* license document attached
... i.e., A record has *exactly one* license document attached



Universal Restrictions on Properties

`allValuesFrom`

The restriction `allValuesFrom` is stated on a property with respect to a class. It means that this property on this particular class has a local range restriction associated with it. Thus **if** an instance of the class is related by the property to a second individual, **then** the second individual can be inferred to be an instance of the local range restriction class

```
:Record a owl:Class ;  
  rdfs:subClassOf [  
    a owl:Restriction ;  
    owl:onProperty :producer ;  
    owl:allValuesFrom :MusicArtist  
  ] .
```

$\text{Record} \sqsubseteq \forall \text{producer. MusicArtist}$

Example of inferences

```
Help! :a :Record ;  
      :producer :GeorgeMartin .  
>>> we infer >>>  
      :GeorgeMartin :a :MusicArtist .
```

```
:Movie a owl:Class ;  
  rdfs:subClassOf [  
    a owl:Restriction ;  
    owl:onProperty :producer ;  
    owl:allValuesFrom :MovieProducer  
  ] .
```

$\text{Movie} \sqsubseteq \forall \text{producer. MovieProducer}$

Example of inferences

```
:JurassicPark :a :Movie .  
              :producer :StevenSpielberg .  
>>> we infer >>>  
      :StevenSpielberg :a :MovieProducer .
```

In other words, the range of `:producer` is:

- `:MusicArtist`, if the subject is a `:Record`
- `:MovieProducer`, if the subject is a `:Movie`

Existential/Universal Restrictions on Properties

allValuesFrom

The restriction *allValuesFrom* is stated on a property with respect to a class. It means that this property on this particular class has a local range restriction associated with it. Thus **if** an instance of the class is related by the property to a second individual, **then** the second individual can be inferred to be an instance of the local range restriction class

```
:Record a owl:Class ;  
  rdfs:subClassOf [  
    a owl:Restriction ;  
    owl:onProperty :producer ;  
    owl:allValuesFrom :MusicArtist  
  ] .
```

“Every record has *only* producers of type MusicArtist”
If the subject is a *Record*,
then every value of *producer* (if it exists) has type *MusicArtist*

$\text{Record} \sqsubseteq \forall \text{producer. MusicArtist}$

someValuesFrom

The restriction *someValuesFrom* is stated on a property with respect to a class. A particular class may have a restriction on a property that at least one value for that property is of a certain type

```
:Album a owl:Class ;  
  rdfs:subClassOf [  
    a owl:Restriction ;  
    owl:onProperty :artist ;  
    owl:someValuesFrom :MusicArtist  
  ] .
```

“Every album has *some* artist of type MusicArtist”
If the subject is a *Album*,
then there exists some value of *artist* that has type *MusicArtist*

$\text{Album} \sqsubseteq \exists \text{artist. MusicArtist}$

Basic Inference Tasks: More Pragmatically..

- Consistency check (Tbox / ABox)
- Infer subclass / subproperty relations
- Inheritance from superclasses
- Classification: infer entity types (based on axioms)
- Infer new relations (e.g., from transitive, symmetric or inverse properties)

About KB Completeness

■ **Open World Assumption** as a default choice

- Close World: what is not proved to be true is assumed to be false
 - E.g., *Is Florence located in USA? NO if not explicitly stated*
 - Default in DBs, and in RDF query processing, e.g., ASK queries
- Open World: what is not proved from the KB is not assumed to be false
 - E.g., *Is Florence located in USA? Can't tell if not explicitly stated*
 - Default in KBs and OWL2

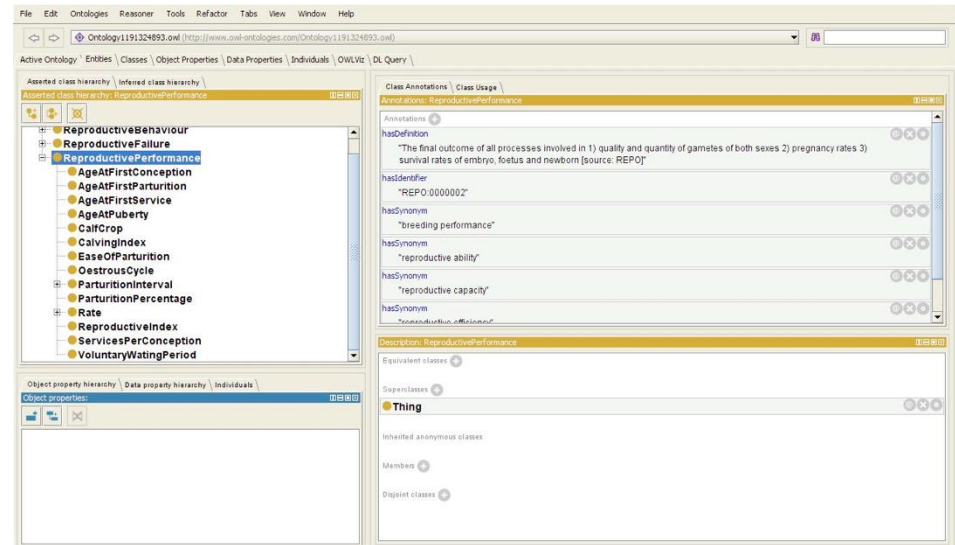
■ **No Unique Name Assumption** as a default choice

- Unique Name Assumption: different names refer to distinct entities
 - E.g., `imdb:Kaya_Wilkins` and `imdb:Brad_Pitt` assumed to be different, but also `imdb:Kaya_Wilkins` and `wd:Okay_Kaya` assumed to be different
- No Unique Name Assumption: different names are not assumed to refer to different entities unless explicitly stated (with predicate `owl:differentFrom`)
 - E.g., `imdb:Kaya_Wilkins` and `imdb:Brad_Pitt` not assumed to be different, but also `imdb:Kaya_Wilkins` and `wd:Okay_Kaya` not assumed to be different

Ontology Editing

State-of-the-art open source editor:
Protegé

- Graphical support for design and editing (for TBox and ABox)
 - Editing and representation with Description Logic syntax and automatic generation of OWL and RDF code
 - Basic checks
 - Visualization Tools



Integration with state of the art
reasoners, rule based systems and
query systems

Download Protegé at:
<https://protege.stanford.edu/>

Start to try it by opening an existing ontology, e.g., the Pizza ontology.

→ “Open from URL” using ”<http://www.cs.ox.ac.uk/isg/ontologies/UID/00793.owl>”

Also check: <https://protege.stanford.edu/support.php>

Example – Cultural Heritage Ontology

- Cultural-ON (Cultural ONtology): Ontologia dei Luoghi della Cultura e degli Eventi Culturali
 - Joint project by MiBACT and ISTC-CNR, ST-Lab
 - Developed with LOD E methodology
 - Documentation Link:
<https://dati.beniculturali.it/cultural-ON/ITA.html>
 - Ontology link: <https://dati.beniculturali.it/cultural-ON/cultural-ON.owl>
 - Follow-up project:
<http://www.iccd.beniculturali.it/it/progetti/4597/arco-architettura-della-conoscenza-ontologie-per-la-descrizione-del-patrimonio-culturale>

DATA SEMANTICS @ DATA SCIENCE @ UNIMIB

